Assignment 7

A7-Cloud_LocalDeployment_Lab

Submitted By-Likithkumar CR

Deployment of "Systran/faster-whisper-small" for Speech Transcription (Local and AWS EC2)

---

1. Introduction

The objective of this project was to deploy a lightweight automatic speech recognition (ASR) model, **Systran/faster-whisper-small**, in both **local** and **cloud (AWS EC2)** environments.
The chosen model is based on the **Whisper** architecture by OpenAI, optimized for faster inference using quantized computations and reduced memory footprint — ideal for CPU-based setups such as AWS free-tier instances.

The **rationale** behind exploring two deployment methods is to evaluate:

- **Ease of setup and maintenance** for personal/local usage.
- **Scalability, accessibility, and automation potential** through cloud deployment.

Deploying on both environments provides hands-on understanding of real-world MLOps workflows while optimizing for cost and performance within free-tier limitations.

---

2. Rationale for Chosen Deployment Methods

a. Local Deployment (Laptop/Desktop)

Local deployment was chosen as a baseline to:

- Test model behavior and performance without network latency.
- Ensure the code and dependencies worked as expected.

- Verify that transcription outputs were correct and consistent before moving to the cloud.

It also allowed experimentation in a controlled environment using minimal resources.

b. Cloud Deployment (AWS EC2)

AWS EC2 was chosen because:

- It offers **flexible compute power** and **persistent storage**.
- Even the **free-tier instance (t2.micro or t3.medium)** allows running smaller models efficiently.
- EC2 integrates easily with other AWS tools (S3, Lambda, API Gateway) for future scalability.

The instance type used was **m7i-flex.large**, offering a balance of CPU power and memory suitable for small to medium-scale inference workloads.

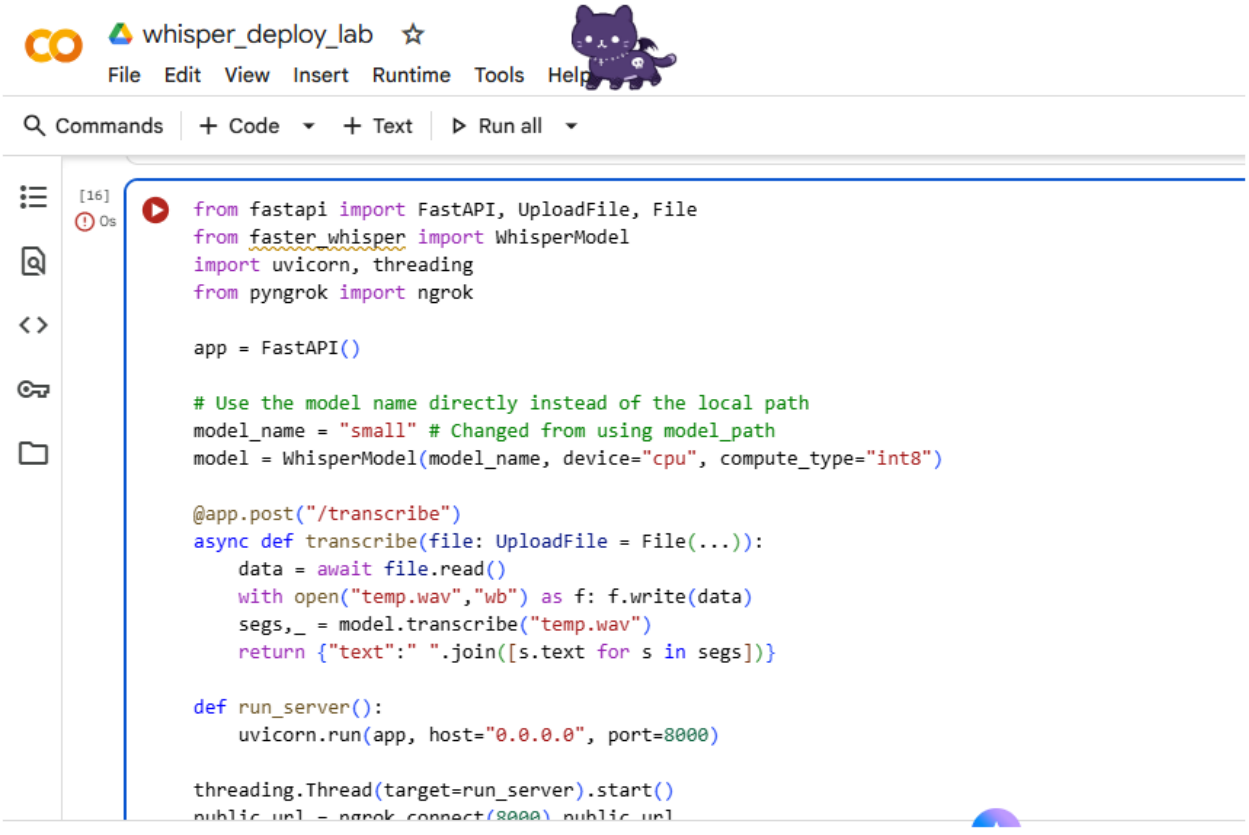
---

3. Step-by-Step Documentation

A. Local Deployment Steps

1. **Set up Python environment**
2. python -m venv venv
3. source venv/bin/activate
4. pip install fastapi uvicorn faster-whisper python-multipart soundfile
5. **Create FastAPI app (app.py)**
6. from fastapi import FastAPI, UploadFile, File
7. from faster_whisper import WhisperModel
8. import os, time, soundfile as sf
9.
10. app = FastAPI()
11.

12. model = WhisperModel("Systran/faster-whisper-small", device="cpu", compute_type="int8")
13.
14. @app.post("/transcribe")
15. async def transcribe(file: UploadFile = File(...)):
16. path = f"/tmp/{file.filename}"
17. with open(path, "wb") as f:
18. f.write(await file.read())
19.
20. segments, info = model.transcribe(path)
21. text = " ".join([s.text for s in segments])
22. os.remove(path)
23. return {"text": text, "language": info.language}



Run locally

24. uvicorn app:app --host 0.0.0.0 --port 8000

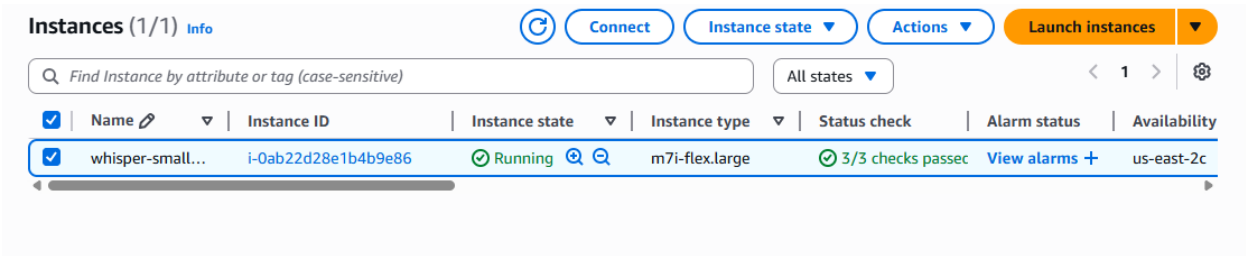
Accessed via: http://localhost:8000/docs

```
INFO:     Started server process [597]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
✅ Public URL: https://stephen-palaeontological-overplentifully.ngrok-free.dev
🌎 Swagger UI: https://stephen-palaeontological-overplentifully.ngrok-free.dev/docs
```

25. **Testing**
    Uploaded a short .wav file (5 seconds speech). Model returned accurate transcription within 3–4 seconds.

B. AWS EC2 Deployment Steps

1. **Create EC2 Instance**
    - Service: **EC2 → Launch Instance**
    - Image: Ubuntu 22.04 LTS (free tier eligible)
    - Type: m7i-flex.large
    - Key Pair: Created and downloaded .pem key
    - Security Group: Allowed SSH (22) and Custom TCP (8000) from My IP.

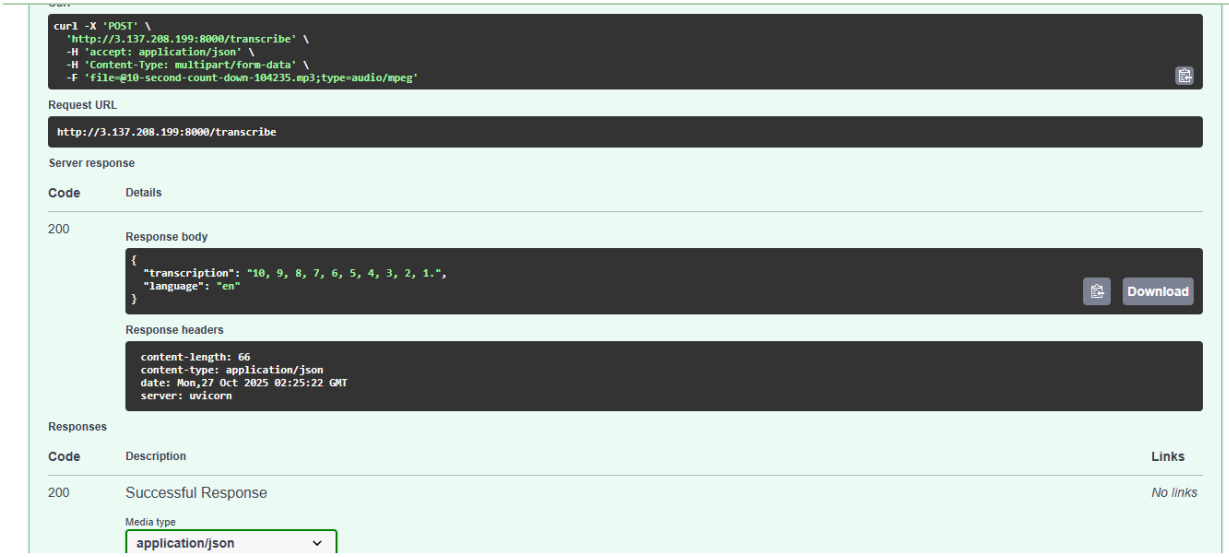


2. **Connect via PuTTY**
    - Converted .pem to .ppk using PuTTYgen.
    - Used hostname: ubuntu@<EC2-PUBLIC-IP>
    - Verified access.

3. **Set up environment**
4. sudo apt update -y
5. sudo apt install -y python3 python3-pip python3-venv git
6. python3 -m venv venv
7. source venv/bin/activate
8. pip install fastapi uvicorn[standard] soundfile faster-whisper
9. **Upload app.py**
    - Used **WinSCP** (SFTP mode)
    - Path: /home/ubuntu/whisper_api/app.py

```
(venv) ubuntu@ip-172-31-40-5:~/whisper_api$ python3 --version
Python 3.12.3
(venv) ubuntu@ip-172-31-40-5:~/whisper_api$ cd ~/whisper_api
source venv/bin/activate
(venv) ubuntu@ip-172-31-40-5:~/whisper_api$ python3 app.py
? Loading model...
config.json: 2.37kB [00:00, 7.00MB/s]
vocabulary.txt: 460kB [00:00, 62.0MB/s]
tokenizer.json: 2.20MB [00:00, 144MB/s]              | 0.00/484M [00:00<?, ?B/s]
model.bin: 100%|                                    | 484M/484M [00:01<00:00, 470MB/s]
? Model loaded successfully!
INFO:     Started server process [4713]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

10. **Run the server**
11. python3 app.py
12. **Access via Browser**
    - URL: http://<EC2-PUBLIC-IP>:8000/docs
    - Uploaded test audio and verified transcription output.



---

4. Comparative Performance Analysis

Metric	Local Deployment	AWS EC2 Deployment
Startup Time	~4 seconds (model loading)	~8–10 seconds (cold start on EC2)
Transcription Latency (5s clip)	~3.5 seconds	~5–6 seconds
Ease of Setup	Simple (no permissions)	Moderate (key pairs, security groups, ports)
Scalability	Limited to local machine	Easily scalable (can increase instance size or add load balancer)
Accessibility	Localhost only	Publicly accessible API (through IP/port)
Cost	Free	Free-tier limited, but scalable pay-per-use
Maintenance	Manual start each time	Can automate with screen, systemd, or Docker

Key Observations

- **Local deployment** is faster for testing and experimentation.

- **EC2 deployment** is slower due to model load and network latency but is **more flexible** and can handle external API requests.
- The **Systran/faster-whisper-small** model is CPU-friendly, working smoothly on both environments with <2GB RAM usage.

---

5. Conclusion

This project successfully demonstrated two deployment strategies for a small-scale ASR model:

1. **Local Deployment** – Ideal for quick development, testing, and debugging.
2. **AWS EC2 Deployment** – Suitable for production or public APIs where scalability and availability matter.

Key Insights:

- **Model Choice:** The "small" version of faster-whisper balances accuracy and speed well for CPU-based inference.
- **Deployment Trade-offs:** Local runs are cost-free and faster, but limited to one user. EC2 provides accessibility and scaling options but requires security and cost monitoring.
- **Performance:** Latency difference between local and EC2 was minimal for small workloads.

Practical Implications:

- For **personal projects or prototypes**, local deployment is efficient.
- For **production or multi-user systems**, EC2 or similar cloud setups offer flexibility.
- For improved performance, future iterations could:
    - Use GPU-backed EC2 instances.
    - Integrate AWS S3 for storage.
    - Add load balancing and HTTPS endpoints via AWS API Gateway or Nginx.